

AI Agents and Agentic AI with Python & Generative AI

Module 1

Agentic AI Concepts

- Introduction
Video - 10 min
- Fixed-Interaction Pattern
Video - 7 min
- The Agent Loop
Video - 8 min
- Knowledge Check: Agent Loop & Fixed Interaction
Practice Assignment - Grade: 100%
- Running the Code Samples in the Course
Reading - 10 min
- Programmatic Prompting for Agents
Ungraded Plug-in - 15 min
- Try Out Programmatic Prompting
Reading - 10 min
- Programmatic Prompting for Agents II
Ungraded Plug-in - 15 min
- Programmatic Prompting for Agents II
Ungraded Plug-in - 15 min
- Programmatic Prompting: Building Blocks for AI Agents
Design - 15 min
- Try Out the Customer Service Agent
Reading - 10 min
- Giving Agents Memory
Ungraded Plug-in - 15 min
- Practicing Programmatic Prompting for Agents (Selected)
Ungraded Plug-in - 15 min
- Adding Structure to AI Agent Outputs
Video - 14 min
- Building Your First Agent
Ungraded Plug-in - 15 min
- AI Agents / Environment Interface
Ungraded Plug-in - 15 min
- AI Agent Feedback and Memory
Ungraded Plug-in - 15 min
- The Agent Loop: From Understanding to Thinking like an Agent
Design - 15 min
- Learning/Task Sharing Connected
Reading - 10 min
- Understanding Agentic AI Concepts
Graded Assignment - 18 min

Module 2

AI Agents, Tools, Actions, & Language

Module 3

GAME: A Conceptual Framework for AI Agents

Module 4

Agent Tool Management

Module 5

Rethinking How Software is Built in the Age of AI Agents

Practicing Programmatic Prompting for Agents (Solution)

Possible Solution for Building a Quasi-Agent

Take a minute to scroll to the bottom to look at the complete solution. Once you are done, come back here and we will walk through the solution.

Understanding the Architecture

Our quasi-agent works through three key steps:

- Initial code generation
- Documentation enhancement
- Test case creation

The magic happens in how we maintain context between these steps, ensuring each builds on the previous results.

Core Components

Let's break down the key pieces:

```
def generate_response(messages: List[Dict]) -> str:
    """Call LLM to get response"""
    response = completion(
        model="gpt4",
        messages=messages,
        max_tokens=1024
    )
    return response.choices[0].message.content
```

This function handles our LLM interactions using OpenAI format. Each message includes a role ("system", "user", or "assistant") and content.

```
def extract_code_block(response: str) -> str:
    """Extract code block from response"""
    if not "```" in response:
        return response
    code_block = response.split("```")[1].strip()
    if code_block.startswith("python"):
        code_block = code_block[6:]
    return code_block
```

The LLM often includes commentary with its code. This function extracts just the code block, making it easier to build upon in subsequent prompts.

The Development Process

The main function, `develop_custom_function()`, orchestrates three phases of development:

Phase 1: Initial Code Generation

```
messages = [
    {"role": "system", "content": "You are a Python expert helping to develop a function."}
]

messages.append(
    {"role": "user",
    "content": "I want a Python function that (function_description). Output the function in a \"python code block\"."}
)
```

We start with a system message establishing the LLM's role, then request initial code based on the user's description.

Phase 2: Documentation Enhancement

```
messages.append(
    {"role": "assistant",
    "content": "\n\npython\n\n\"\"\"Initial function\n\n\"\"\""}
)

messages.append(
    {"role": "user",
    "content": "Add comprehensive documentation to this function..." }
)
```

Notice how we feed back the code but strip any commentary. This keeps the LLM focused on just the code structure.

Phase 3: Test Case Generation

```
messages.append(
    {"role": "assistant",
    "content": "\n\npython\n\n\"\"\"documented_function\n\n\"\"\""}
)

messages.append(
    {"role": "user",
    "content": "Add unittest test cases for this function..." }
)
```

Again, we maintain clean context by showing only the documented code.

Memory Management Through Message History

The key insight is how we manage "memory" through the messages list. Each step builds on previous responses, but we carefully control what the LLM sees:

- We only show the code, not the commentary
- Each message provides specific instruction for the next enhancement
- The context builds progressively through the message history

For example, when adding documentation, the LLM sees:

- It's a Python expert (system message)
- The original code (previous response)
- The request for documentation (current task)

This focused context helps ensure consistent, high-quality output.

Usage Example

Here's how it works in practice:

```
>>> function_code, tests, filename = develop_custom_function()
What kind of function would you like to create?
Example: 'A function that calculates the factorial of a number'
Your description: Calculate fibonacci sequence up to n

=== Initial function ===
def fibonacci(n):
    if n == 0:
        return 1
    elif n == 1:
        return 1
    sequence = [0, 1]
    while len(sequence) < n:
        sequence.append(sequence[-1] + sequence[-2])
    return sequence

=== Documented function ===
[... function with added documentation ...]

=== Test Cases ===
[... unittest test cases ...]

Final code has been saved to calculate_fibonacci_sequence_up.py
```

Learning from This Design

This quasi-agent teaches us several important lessons about building LLM-powered systems:

- Prompt Chaining:** Breaking complex tasks into sequential steps makes them more manageable.
- Context Management:** Carefully controlling what the LLM sees helps maintain focus and consistency.
- Output Processing:** Having robust ways to extract and clean LLM output is crucial.
- Progressive Enhancement:** Building features iteratively (code -> docs -> tests) creates better results than trying to do everything at once.

These principles apply even when building more complex, fully agentic systems.

Complete Solution

```
from IPython import completion
from typing import List, Dict
import sys

def generate_response(messages: List[Dict]) -> str:
    """Call LLM to get response"""
    response = completion(
        model="gpt4",
        messages=messages,
        max_tokens=1024
    )
    return response.choices[0].message.content

def extract_code_block(response: str) -> str:
    """Extract code block from response"""
    if not "```" in response:
        return response
    code_block = response.split("```")[1].strip()
    # Check for "python" at the start and remove
    if code_block.startswith("python"):
        code_block = code_block[6:]
    return code_block

def develop_custom_function():
    # Get user input for function description
    print("What kind of function would you like to create?")
    print("Example: 'A function that calculates the factorial of a number'")
    print("Your description: ", end="")
    function_description = input().strip()

    # Initialize conversation with system prompt
    messages = [
        {"role": "system", "content": "You are a Python expert helping to develop a function."}
    ]

    # First prompt - Basic function
    messages.append(
        {"role": "user",
        "content": "I want a Python function that (function_description). Output the function in a \"python code block\"."}
    )
    initial_function = generate_response(messages)

    # Parse the response to get the function code
    initial_function = extract_code_block(initial_function)

    print("\n=== Initial Function ===")
    print(initial_function)

    # Add assistant's response to conversation
    # Notice that I am purposely leaving it to forget its commentary and just see the code so that
    # it appears that it is always outputting just code.
    messages.append({"role": "assistant", "content": "\n\npython\n\n\"\"\"Initial function\n\n\"\"\""})

    # Second prompt - Add documentation
    messages.append(
        {"role": "user",
        "content": "Add comprehensive documentation to this function, including description, parameters, \"return value, examples, and edge cases. Output the function in a \"python code block\"."}
    )
    documented_function = generate_response(messages)
    documented_function = extract_code_block(documented_function)
    print("\n=== Documented Function ===")
    print(documented_function)

    # Add documentation response to conversation
    messages.append({"role": "assistant", "content": "\n\npython\n\n\"\"\"documented_function\n\n\"\"\""})

    # Third prompt - Add test cases
    messages.append(
        {"role": "user",
        "content": "Add unittest test cases for this function, including tests for basic functionality."}
    )
    # Edge cases, error cases, and various input scenarios.
    # Output the code in a \"python code block\"
    test_cases = generate_response(messages)
    # We will likely run into random problems here depending on if it outputs 202 the test cases or the
    # test cases AND the code. This is the type of issue we will learn to work through with agents in the course.
    test_cases = extract_code_block(test_cases)
    print("\n=== Test Cases ===")
    print(test_cases)

    # Generate filename from function description
    filename = function_description.lower()
    filename = ".json" if "in filename" in filename or "isinstance" in filename else filename.replace(" ", "_")[:30] + ".py"

    # Save final version
    with open(filename, "w") as f:
        f.write(documented_function + "\n\n" + test_cases)

    return documented_function, test_cases, filename

if __name__ == "__main__":
    function_code, tests, filename = develop_custom_function()
    print(f"Final code has been saved to {filename}")
```

Expand

Mark as completed